

C++ PROGRAMMING

A S S I G N M E N T 1

Constructors · Function Overloading · Default Arguments · Array of Objects

SUBJECT
C++ Programming

TYPE
Assignment 1

QUESTIONS
6 Programs

Q1**Differences Between #define and const Keyword + WAP Using Constant Qualifier****CONCEPT****i**

#define is a preprocessor directive that performs text substitution before compilation with no type checking. const is a typed, scoped variable enforced by the compiler with full type safety.

COMPARISON TABLE

Feature	#define	const Keyword
Processed by	Preprocessor	Compiler
Type Safety	None (text substitution)	Full type checking
Scope	Global (till #undef)	Block/function scope
Memory	No memory allocated	Memory allocated
Debugging	Harder (no type info)	Easier (visible in debugger)
Syntax	#define PI 3.14	const float PI = 3.14;

PROGRAM CODE — WAP USING CONST

```
#include <iostream>
using namespace std;

#define GRAVITY 9.8f           // preprocessor constant (no type)

int main() {
    const float PI = 3.14159f; // compiler-typed constant
    const int MAX_DAYS = 365;
    float radius;

    cout << "Enter radius of circle: ";
    cin >> radius;

    cout << "Area          = " << PI * radius * radius << endl;
    cout << "Circumference= " << 2 * PI * radius << endl;
    cout << "Max Days/Year= " << MAX_DAYS << endl;
    cout << "Gravity       = " << GRAVITY << endl;

    // PI = 3.14; // ERROR: const cannot be modified
    return 0;
}
```

SAMPLE OUTPUT

```
Enter radius of circle: 5
Area          = 78.5397
Circumference= 31.4159
Max Days/Year= 365
Gravity       = 9.8
```

★ KEY POINTS

- ▶ `#define` replaces text blindly — e.g. `#define SIX 1+5` → `2*SIX = 2*1+5 = 7` (not 12!).
- ▶ `const` respects operator precedence and data types — always prefer `const` over `#define`.
- ▶ `const` variables are visible in the debugger; `#define` symbols are stripped before compilation.

Q2 WAP with User-Defined Functions: Readdata(), Adddata(), Displaydata()**CONCEPT**

i

User-defined functions break a program into modular, reusable units. Each function has a single responsibility: reading input, processing data, or displaying output — following the separation of concerns principle.

◆ PROGRAM CODE

```
#include <iostream>
using namespace std;

float a, b, c, sum; // global variables shared across functions

// Function 1: Read three floating-point numbers
void Readdata() {
    cout << "Enter three floating-point numbers:" << endl;
    cout << "  Number 1: "; cin >> a;
    cout << "  Number 2: "; cin >> b;
    cout << "  Number 3: "; cin >> c;
}

// Function 2: Calculate the sum
void Adddata() {
    sum = a + b + c;
}

// Function 3: Display the result
void Displaydata() {
    cout << "\n--- Result ---" << endl;
    cout << "Numbers : " << a << ", " << b << ", " << c << endl;
    cout << "Sum      : " << sum << endl;
}

int main() {
    Readdata(); // call 1: read input
    Adddata();  // call 2: compute sum
    Displaydata(); // call 3: display output
    return 0;
}
```

▶ SAMPLE OUTPUT

```
Enter three floating-point numbers:
  Number 1: 12.5
  Number 2: 7.3
  Number 3: 4.2

--- Result ---
Numbers : 12.5, 7.3, 4.2
Sum      : 24
```

★ KEY POINTS

- ▶ Global variables (a, b, c, sum) allow all three functions to share state without parameters.
- ▶ Alternatively, functions can use pass-by-reference for cleaner design.
- ▶ Each function has exactly one responsibility — this is the Single Responsibility Principle.

Q3**WAP to Calculate Volume of Cube, Rectangular Box & Cylinder Using Class, Objects and Function Overloading****CONCEPT****i**

Function overloading inside a class lets a single method name (volume) handle different shapes based on argument count/type. The compiler selects the correct version at compile time — static polymorphism.

◆ PROGRAM CODE

```
#include <iostream>
#define PI 3.14159f
using namespace std;

class Volume {
public:
    // Overload 1: Cube — one side
    float calculate(float side) {
        return side * side * side;
    }

    // Overload 2: Rectangular Box — l x b x h
    float calculate(float l, float b, float h) {
        return l * b * h;
    }

    // Overload 3: Cylinder — PI * r^2 * h (int dummy differentiates)
    float calculate(float radius, float height, int) {
        return PI * radius * radius * height;
    }
};

int main() {
    Volume v;
    cout << "Volume of Cube (side=3)           : " << v.calculate(3.0f)           <<
endl;
    cout << "Volume of Rect Box (4 x 3 x 2)      : " << v.calculate(4.0f, 3.0f, 2.0f) <<
endl;
    cout << "Volume of Cylinder (r=5, h=10)      : " << v.calculate(5.0f, 10.0f, 0) <<
endl;
    return 0;
}
```

▶ SAMPLE OUTPUT

```
Volume of Cube (side=3)           : 27
Volume of Rect Box (4 x 3 x 2)      : 24
Volume of Cylinder (r=5, h=10)      : 785.397
```

★ KEY POINTS

- ▶ The dummy int in the cylinder overload distinguishes it from the rectangular box version.
- ▶ Overload resolution is done at compile-time based on number and types of arguments.

- ▶ Encapsulating volume logic in a class allows creating multiple independent Volume objects.

Q4**WAP with Class Having Two Data Members and Two Constructors (Default + Parameterized)****CONCEPT****i**

A constructor is a special member function called automatically when an object is created. A default (no-arg) constructor initializes members to zero/default values; a parameterized constructor accepts arguments to set them explicitly.

◆ PROGRAM CODE

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    float length;
    float breadth;

public:
    // Constructor 1: Default (no arguments) – initializes to 0
    Rectangle() {
        length = 0;
        breadth = 0;
        cout << "[Default Constructor called] length=0, breadth=0" << endl;
    }

    // Constructor 2: Parameterized – initializes to given values
    Rectangle(float l, float b) {
        length = l;
        breadth = b;
        cout << "[Parameterized Constructor called] l=" << l << ", b=" << b << endl;
    }

    void display() {
        cout << " Length=" << length << " Breadth=" << breadth;
        cout << " Area=" << length * breadth << endl;
    }
};

int main() {
    Rectangle r1; // calls Default Constructor
    Rectangle r2(6.0f, 4.0f); // calls Parameterized Constructor
    Rectangle r3(10.0f, 3.5f);

    cout << "\nObject r1 → "; r1.display();
    cout << "Object r2 → "; r2.display();
    cout << "Object r3 → "; r3.display();
    return 0;
}
```

▶ SAMPLE OUTPUT

```
[Default Constructor called] length=0, breadth=0
[Parameterized Constructor called] l=6, b=4
[Parameterized Constructor called] l=10, b=3.5
```

```
Object r1 → Length=0    Breadth=0    Area=0
Object r2 → Length=6    Breadth=4    Area=24
Object r3 → Length=10   Breadth=3.5  Area=35
```

★ KEY POINTS

- ▶ Constructors have the same name as the class and no return type (not even void).
- ▶ The cout inside constructors confirms WHICH constructor is invoked during object creation.
- ▶ Constructor overloading follows the same rules as function overloading.

Q5

Modify Q4 — Default Value for Second Argument of Parameterized Constructor

CONCEPT

i

Default arguments can be applied to constructor parameters just like regular functions. When a default is provided for the second parameter, the constructor can be called with only one argument — the second defaults automatically.

◆ PROGRAM CODE

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    float length;
    float breadth;

public:
    // Default Constructor
    Rectangle() {
        length = 0;
        breadth = 0;
        cout << "[Default Constructor] l=0, b=0" << endl;
    }

    // Parameterized Constructor – breadth defaults to 1.0f
    Rectangle(float l, float b = 1.0f) {
        length = l;
        breadth = b;
        cout << "[Param Constructor] l=" << l << ", b=" << b << endl;
    }

    void display() {
        cout << " Length=" << length << " Breadth=" << breadth;
        cout << " Area=" << length * breadth << endl;
    }
};

int main() {
    Rectangle r1;                // Default Constructor
    Rectangle r2(5.0f);          // Only 1 arg – breadth defaults to 1.0
    Rectangle r3(8.0f, 3.0f);    // Both args provided

    cout << "\nr1 → "; r1.display();
    cout << "r2 → "; r2.display();
    cout << "r3 → "; r3.display();
    return 0;
}
```

▶ SAMPLE OUTPUT

```
[Default Constructor] l=0, b=0
[Param Constructor] l=5, b=1
[Param Constructor] l=8, b=3
```

r1 →	Length=0	Breadth=0	Area=0
r2 →	Length=5	Breadth=1	Area=5
r3 →	Length=8	Breadth=3	Area=24

★ KEY POINTS

- ▶ Default arguments in constructors must be trailing parameters (right to left).
- ▶ r2(5.0f) calls the parameterized constructor with breadth defaulting to 1.0.
- ▶ Providing default constructor arguments reduces the need for multiple constructors.

Q6 Implement Array of Objects**CONCEPT****i**

An array of objects is a collection where each element is an instance of a class. Just like arrays of int or float, each object occupies a slot, and member functions are accessed via the index and dot operator.

◆ PROGRAM CODE

```
#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    string name;
    int    rollNo;
    float  marks;

public:
    // Default constructor required for array declaration
    Student() {
        name    = "Unknown";
        rollNo  = 0;
        marks   = 0.0f;
    }

    void setData(string n, int r, float m) {
        name = n; rollNo = r; marks = m;
    }

    void display(int index) {
        cout << "    [" << index << "]" << name;
        cout << "    Roll: " << rollNo;
        cout << "    Marks: " << marks << "%" << endl;
    }

    float getMarks() { return marks; }
};

int main() {
    const int SIZE = 4;
    Student students[SIZE];    // array of 4 Student objects

    // Initialize each object
    students[0].setData("Alice", 101, 92.5f);
    students[1].setData("Bob", 102, 78.0f);
    students[2].setData("Charlie", 103, 85.5f);
    students[3].setData("Diana", 104, 95.0f);

    cout << "===== Student Records =====" << endl;
    for (int i = 0; i < SIZE; i++)
        students[i].display(i + 1);

    // Find highest scorer
    int top = 0;
    for (int i = 1; i < SIZE; i++)
        if (students[i].getMarks() > students[top].getMarks())
```

```
        top = i;

    cout << "\nTop Scorer: ";
    students[top].display(top + 1);
    return 0;
}
```

▶ SAMPLE OUTPUT

```
===== Student Records =====
[1] Alice      Roll: 101  Marks: 92.5%
[2] Bob        Roll: 102  Marks: 78%
[3] Charlie    Roll: 103  Marks: 85.5%
[4] Diana      Roll: 104  Marks: 95%

Top Scorer:
[4] Diana      Roll: 104  Marks: 95%
```

★ KEY POINTS

- ▶ Declaring an array of objects (Student s[4]) calls the default constructor for each element.
- ▶ A default constructor is mandatory when declaring arrays of objects — otherwise compilation fails.
- ▶ Loop with dot-notation (students[i].display()) iterates over all objects efficiently.